Forecasting first diagnoses from synthetic EHR timelines

Research intern coding test: technical report

[AUTHOR NAME]

4 November 2026

Links. Code and documentation: https://github.com/Reaper8202/Research_Intern_Coding_Test. Trained model package: https://huggingface.co/Reaper8202/ehr-forecast. wandb project (training, tuning, and val-curve runs): https://wandb.ai/derek-wu820-university-of-toronto/ehr-forecast.

Summary

The submission is a blend of three models, all trained from scratch on the 2,791 training patients: fifteen small decoder-only transformers (2 layers, width 128, about 738k parameters each; three seeds times five cross-validation folds, self-pretrained on pre-anchor event sequences and then fine-tuned on the 40 labels) and two LightGBM models on code counts, one of which also sees the latest value of each numeric observation. Their logits are averaged, and a per-target Platt map fitted on training out-of-fold predictions scales the result. On the official validation split (365 patients) the blend scores macro AUROC **0.767 [0.746, 0.788]** and mAP **0.258 [0.234, 0.268]**. The transformer ensemble alone gets 0.751 / 0.209, and LightGBM on counts alone 0.762 / 0.239. The test score is the organisers' to compute.

The transformer is the part I built and studied, and it is what most of this report is about. The blend came at the end, from a rule written before its score existed: in nested cross-validation it beat both of its parts with intervals clear of zero. Most of the effort went into the pipeline rather than the models. The truncation rule was checked against the released test files. Column allowlists keep future-bearing fields out. Every model shares one prior-condition mask. Every choice was made by five-fold cross-validation on train, and each of the three tuning rounds had its budget committed before its transformer runs started (round 3's stacking rule was committed while the stacking job was already running; section 2 says so). An independent read-only session audited every data boundary, a 74-criterion audit closed the project (`docs/audit_criteria.md`, `reports/audit/`), and official val was read three times in all, each read recorded.

1. Representation

One unit of input is one clinical event, written as a single token `source:kind:code` with its value folded in. A patient's sequence looks like this:

```
[SEX:F]  hx:<older conditions> ...  [SEP]  <events, oldest first, one token each>
```

Source	Token	Example
conditions, medications, care plans, allergies, devices	`src:start:code`, `src:stop:code`	`med:start:310798`
procedures, immunizations	`proc:code`, `imm:code`	`proc:430193006`
numeric observations	`obs:code:unit:q<bin>`	`obs:8480-6:mm[Hg]:q7`
text observations	`obs:code:<normalized value>`	`obs:72166-2:never_smoker`
imaging studies, encounters	`img:modality|site`, `enc:class:code`	`enc:wellness:162673000`
genotype (D34)	`[DNA]`, `dna:<site>:<alleles>`	one token per carried variant
brain MRI (D34)	`[IMG]`	embedding replaced by a 3-D CNN encoding

Fields and modalities. Every CSV table is read through a column allowlist. I dropped, on purpose: death date, lifetime healthcare expenses and coverage, marital status, addresses, all cost and payer columns, medication dispense counts and total cost, and device UDIs. The dispense counts were the sneaky one. In the released test files, 873 of the 1,049 medications that continue past the anchor have more dispenses than months elapsed, so that column carries future refills even after the organisers' truncation. Device UDIs encode manufacture and expiry dates. Race and ethnicity are not used (D15; a LightGBM check showed they cost nothing measurable).

Three observation codes, DALY, QALY, and QOLS, are removed as well (D36). They are simulator quality-of-life scores recorded once a year from 2011, so their counts say how far before 2021 the record ends, which is the anchor year. Their counts predict an anchor before 2015 with AUROC 0.999 (`reports/dataset/clock_scan.txt`). That is not a leak; every row is pre-anchor. It is a validity problem: the model would be reading how the anchor was built, not anything about the patient. Removing them cost about 0.014 CV AUROC in the transformer and 0.005 in the final blend, and it probably costs test score too, since the test files contain the codes.

Genomics and the brain-MRI DICOM volumes are in, as a test of the optional modalities after an earlier decision to leave them out. The test came back null and the tokens stayed, which is reported as such. Coverage is thin: genotype files exist for 396 training and 2 test patients, and a pre-anchor scan for 200 training and 28 test patients. A scan enters as one token only when its study date is before the anchor day. In CV they add nothing (+0.0005 AUROC overall; on the 541 training patients whose inputs actually change, -0.003 [-0.020, +0.015]). The volumes are near-identical simulated brains (block-mean correlation 0.995 to 0.999 between patients). The audit (section 3) found that simply having a genotype file or a scan correlates with outcomes in this dataset: genotype files for 14.2% of train patients against 0.6% of val and test, with 2.11 against 1.83 positive labels per patient (`reports/dataset/modality_presence.json`). So the presence tokens could act as a proxy. The CV result says the model doesn't use them, and only 2 test patients have genotypes.

Continuous values. Each numeric observation is binned into quantiles fitted per code and unit on the fold's fit patients only (up to 10 bins, merged where boundaries coincide), and the bin index goes into the token. Units stay separate. No unit conversions or clinical thresholds come from outside. A value token needs at least five fit patients; otherwise it maps to `obs:code:other_value`. Before committing to bins I checked them against raw values in LightGBM: 0.243 against 0.244 mAP, no material loss. The LightGBM part of the blend uses the continuous latest value directly, with a missing flag and the log of days since the measurement.

Time between events. Every token carries the patient's age at that event, to the day, as `age/100` plus sine and cosine terms over eight periods from 0.5 to 200 years, mapped by a learned linear layer and added to the token embedding. So the model can read the gap between any two tokens. Rotary position encoding gives token order. The prediction head also gets the exact time from the last event to the anchor, age at anchor, sex, and a missing-history flag. Explicit gap tokens were planned as an experiment and I didn't reach them within the budget.

Context and summary. Sequences are capped at 1,024 tokens. The most recent complete event days are kept. Conditions first diagnosed before the kept window become `hx:` summary tokens (up to 128) at the front, so an old diagnosis is never lost. Under the final tokenizer about 4.6% of training patients are truncated, but they are the densest histories: 15 to 18% of all event tokens of the fit patients fall outside the window (24.6% on held-out fold 0, whose dense histories are longer), mostly medication refills and encounters. Exact refills (a stop and a restart of the same drug on the same day; 160,150 of them in fold 0) are merged first. Contexts of 512 and 2,048 were screened on fold 0 and did not help.

Leakage controls. Train and val timelines are cut with the same rules the released test files follow: keep rows dated before the anchor day, blank any `STOP` on or after it. I chose the rule among four candidates by matching the distribution of the gap from last event to anchor in the test files (D1). The pipeline stops if any table holds a date on or after an anchor. A test adds a future record and checks that every prediction is unchanged. Labels come from the full record for train and val patients; the prediction interface never sees them. The vocabulary, bins, folds, self-pretraining windows, LightGBM vocabularies and value medians, stacking weights, and Platt maps are all fitted inside each fold on fit patients only.

2. Model and training

Architecture. A decoder-only transformer: pre-LayerNorm blocks, causal multi-head attention, RoPE, GELU MLP at 4x width, token embedding plus the age projection. Two layers, width 128, 4 heads, a vocabulary of about 1,930 tokens, 738k parameters in all. The pretraining head predicts the next token with weights tied to the embedding, and input-only symbols are masked out of the output distribution. The prediction head concatenates the last clinical-event state with the mean state over all clinical events (the `last_mean` readout adopted in round 2) and the four head features, applies dropout, and maps to 40 logits with independent sigmoids. The MRI encoder is a four-layer 3-D CNN (81k parameters), random init, on a 64-cubed volume.

Objectives. Self-pretraining is next-token cross-entropy over clinical event positions, on each fit patient's history strictly before the real anchor, one random full window per patient per epoch; target-diagnosis tokens weigh 5 in the loss (round 2). Fine-tuning is unweighted binary cross-entropy over the unmasked patient-target cells. A target the patient already has is masked out of the loss and set to probability 0 at prediction. The loss is normalised by the cell count across gradient-accumulation micro-batches. Each fit patient also gives one extra supervised example per epoch from an earlier anchor five years back (round 2).

Hyperparameters (`configs/experiments/v2_c1.toml`): AdamW throughout. Pretraining: lr 0.001, betas (0.9, 0.95), weight decay 0.1, 10% warm-up then cosine to 0.1 of peak, batch 32 patients, 30 epochs, patience 3 on fixed held-out windows. Every run reached epoch 30 with held-out loss still falling slowly (fold 0: 1.449). Fine-tuning: head lr 0.001, body lr 0.0001, weight decay 0.01, gradient clip 1.0, batch 32, dropout 0.3 in body and head, up to 30 epochs, patience 5 on held-out masked macro AUROC; best epochs ranged from 3 to 13. One fold (pretraining plus fine-tuning) takes about 20 minutes on a 16 GB M-series Mac (MPS; the resume-equality test runs on CPU), and the 15

submitted members took about 4.5 hours in all. Everything is float32. Two code defaults are not in the config: fine-tuning AdamW betas (0.9, 0.999) and a constant fine-tuning learning rate.

Three training methods, same folds, before any tuning. S trains from random initialisation with supervision only. H keeps the pretrained body fixed and trains the head. F fine-tunes the pretrained body and the head together. F won on every fold: mean fold AUROC 0.760 against 0.744 (S) and 0.742 (H); pooled paired difference F minus S +0.020 [+0.013, +0.028]. So self-pretraining helps, and a fixed body doesn't carry enough on its own. The comparison was made on `small-v1` and `small-v1-mm`; I did not re-run S for the final recipe.

Tuning, pre-registered. Three rounds. Each was written to a budget file and committed before its transformer runs started, with one adoption rule: mean AUROC gain of at least 0.005, higher on at least 4 of 5 folds, mAP loss of at most 0.005. Round 1 (11 trials) adopted dropout 0.3; a lower body learning rate, weight decay 0.1, an age-sex head, 60 pretraining epochs, and contexts 512 and 2,048 all failed. Round 2 (12 trials, after the clock codes were removed) adopted the combination of a pooled readout, extra anchors, and target-weighted pretraining; a count branch, per-target loss weights, and dropout 0.5 failed. Full-timeline pretraining (reading fit patients' post-anchor events) passed the rule and was **not** adopted, because the task says to truncate at the anchor. Three seeds of the winner were trained and averaged. Round 3 tested the one architecture dimension never varied, depth and width: a 4-layer, width-192 model (2.26M parameters) reached a slightly lower pretraining loss but scored 0.758 against 0.772 on the fold-0 screen, so the small model stayed.

One correction from the final audit. Round 3's stacking and calibration rules were committed at 21:18 on 3 November while the stacking job was already running and had written part of its output (its per-target file at 21:04, its summary at 21:19). So those two rules were written with point estimates possibly in view. The architecture screen started after the commit. The rules were not changed afterwards, and the adopted blend passes them by margins well outside the intervals (D38). The full register with every score is in `docs/decisions.md` (D35, D36, D38) and `reports/transformer/`.

Comparison models and the blend. Alongside the transformer I kept an age-sex incidence table (the epidemiological floor, with and without the "already has it" flag), L2 logistic regression on log code counts, and LightGBM on counts and on counts plus latest values, each with settings chosen by the same folds (LightGBM's grid picked its simplest corner, 4 leaves and 100 trees). In training CV the transformer ensemble and LightGBM counts were within noise on AUROC (0.761 against 0.764 pooled out of fold), and LightGBM was ahead on mAP. Round 3 then tested output stacking with nested folds: for each fold, the stacker is fitted on the other four folds' out-of-fold predictions. The parameter-free mean of logits beat the transformer ensemble by +0.014 AUROC [+0.009, +0.019] and LightGBM counts by +0.011 [+0.007, +0.014] (1,000 resamples; mean fold AUROC 0.780, mAP 0.236). A learned per-target logistic stacker gained only 0.001 more, so the pre-registered rule kept the fixed blend. Per-target Platt scaling of the blend, also nested, lowered Brier on 4 of 5 folds and macro expected calibration error from 0.013 to 0.010, with per-target AUROC unchanged by construction (a non-positive fitted slope falls back to the identity).

Curves. Training loss, learning rate, and gradient norm are logged per step, and held-out fold metrics per epoch, for every transformer run in the wandb project above. The 15 members are the runs `small-v2-c1-fold<k>-F`, `small-v2-c1-seed2-fold<k>-F`, and `small-v2-c1-seed3-fold<k>-F` (ids in `reports/final/manifest.json`). Figure 1 shows the requested validation curves: every saved epoch checkpoint of the 15 members, scored on the official val split after training inside the frozen bundle. The epochs selected on the held-out training folds (dots) sit near but mostly before the val optimum. Val would have picked a later epoch in 13 of 15 members and gained 0.008 AUROC on average. I didn't make that choice.

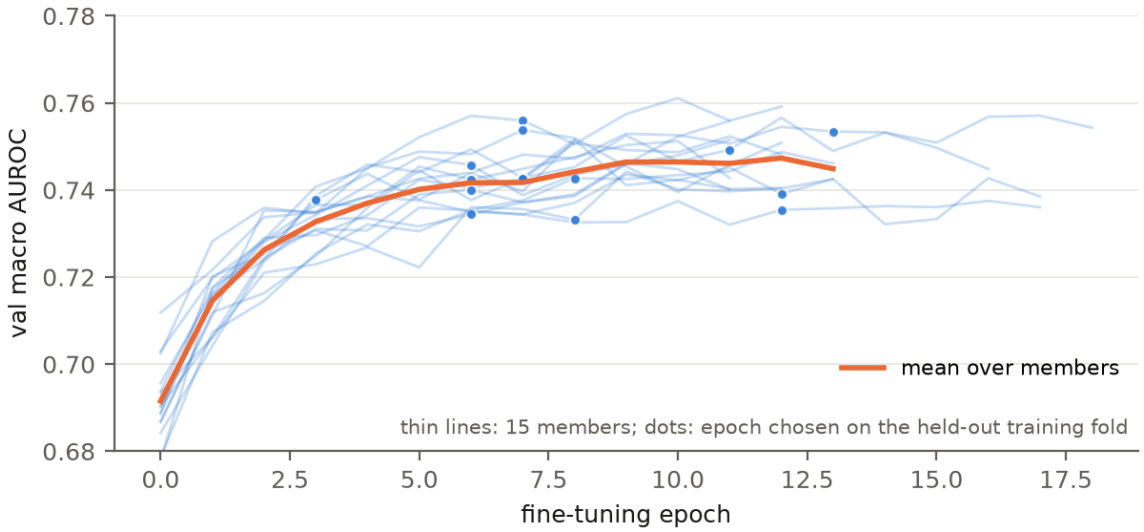


Figure 1: Official val macro AUROC of every saved fine-tuning checkpoint. Thin lines: the 15 ensemble members; dots: the epoch each member's held-out training fold selected; the orange line is the mean over members at that epoch.

Evaluation protocol. Five folds of the training split (plain shuffled K-fold, seed 20261028, no stratification; the rarest target has 4 to 6 positives per held-out fold; patients never shared across folds). One out-of-fold prediction per patient, from the models that did not fit on them. Macro AUROC and mAP over 40 targets with the common prior mask, plus Brier score, with 95% patient-level BCa bootstrap intervals (1,000 resamples) and paired bootstraps for differences. Official val was scored by `make val-score`, which records every artifact, data, code, and lock-file hash at freeze, refuses to run on a changed tree or artifact, writes a start record before any metric and a completion record after, and refuses a second run without a documented reason. It ran twice. Attempt 1 scored the transformer ensemble (0.751 / 0.209) before the blend existed. Attempt 2 scored the blend after round 3 was decided on training CV. Both records are kept (`reports/val/`, `reports/val/attempts/`). Before the bundle existed, the step-2 baselines (logistic regression and the incidence table) had scored val once, on 28 and 29 October. So the val labels were read three times in all. One more thing to be plain about: attempt 1 had shown LightGBM ahead of the transformer on val mAP, and round 3, which added LightGBM to the submission, was opened after that. Its rules used training CV only, but attempt 2 is a second look at the same 365 patients, not an independent check. The hidden test set is the independent check.

3. AI workflow

Claude Code wrote most of the code, the tests, and the planning documents. I set the objective and the constraints, approved or rejected each decision that touched the data or the protocol, and read every result before the next step started. The division of work is in `docs/ai_workflow.md` and the reasoning behind each decision in `docs/decisions.md`.

What kept the AI's work honest: the plan was written before the code, and each revision says what was known when it was made. 199 tests include mutation-style checks (changing < to <= in the anchor rule fails a test; a future record must leave predictions unchanged; a resumed run must match an uninterrupted one; adding held-out rows must not change the fitted vocabulary). The

adoption rule was fixed before the first transformer result. Every experiment block was committed before its scores. And an independent read-only audit by a second AI session checked every data boundary (truncation, allowlists, labels, tokenizer, extra anchors, pretraining windows, early stopping, modalities, LightGBM, evaluation, test split) with file and line references and found no leak.

Problems the review caught. Fold-4 pretraining filled the disk with swap four times because the MPS allocator fragmented across variable-length batches; releasing cached memory each step fixed it, changed no computation, and made training 3 to 6 times faster. LightGBM and PyTorch each load an OpenMP runtime and crashed in one process, so LightGBM now runs in its own process and the test suite pins the import order. The first bootstrap intervals used the percentile method, which for rare targets barely contained the point estimate; BCa replaced them. A preflight audit flagged its own report files as a dirty tree. And a first Platt-scaling fit let five near-chance targets take a negative slope, which would have reversed their ranking; a monotonicity test caught it, and the fit now falls back to the identity. [Derek: add anything else here, including any use of Codex CLI.]

4. Interpretation

Where the model does well. Seven targets are above 0.95 AUROC for the blend (six of them for the transformer and LightGBM counts as well): prostate neoplasm, carcinoma in situ and metastasis of the prostate, normal pregnancy, pneumonia, obesity, and the pregnancy-complication code. Each follows a visible Synthea pathway (PSA tests and rectal exams, oral contraceptives, dementia care plans, BMI observations). A leakage review of these scores found pre-anchor causes, not leaks. The transformer's clearest wins over LightGBM are the paired brain targets, silent micro-haemorrhage and abnormal gait (0.905 against 0.832 on val; the two labels are identical for every patient because one module emits both), and stroke. Figure 2 shows every target for the final blend and its two parts.

Where it does poorly, and why. Nine targets are below 0.6 for the blend (seven of them for the transformer and LightGBM counts too, five for all six models scored): whiplash, lacerations, wrist and ankle injuries, clavicle fracture, sinusitis, and strep throat. Synthea generates these acute events with little dependence on the prior record, so no history model can rank them, and the age-sex table is at chance too. The transformer's largest losses to LightGBM are the count-driven conditions: osteoporosis, recurrent rectal polyp, myocardial infarction, the three lung-cancer codes, and pathological fracture, on val and in CV alike. In Synthea their risk grows with how many of a specific event has happened (screening visits, bone-density tests, nodule follow-ups). A count feature states that directly. The transformer has to infer it from a window that drops 15 to 18% of event tokens for exactly the dense histories where counts are high. The count branch tested in round 2 didn't close the gap, a longer context didn't help on fold 0, and that is why the blend helps: the two models fail on different targets. On val the blend is above the transformer on 29 of 40 targets and above LightGBM on 17. It recovers most of the transformer's count-driven losses (myocardial infarction 0.927 against 0.827 and 0.904, osteoporosis 0.883 against 0.804 and 0.895, pathological fracture 0.911 against 0.847 and 0.909) while keeping most of the transformer's lead on the brain pair (0.892 against 0.832). In CV, the better of the two per target would reach 0.778 against 0.774 for the blend.

Anchor era. Anchors are five years before the last encounter, so an anchor before 2015 almost always means the patient died by 2020 (1,155 of 1,219 training patients), and those patients have more new diagnoses (2.37 against 1.48). Scores split by era are similar for every model. An era signal remains in the clinical codes and age even without the clock codes (AUROC 0.876 for predicting the era on fold 0; `reports/dataset/clock_scan.txt`). It can't be removed without removing history, so it is reported.

Calibration. Prior-condition cells are exact zeros. The raw blend under-predicts slightly in the middle

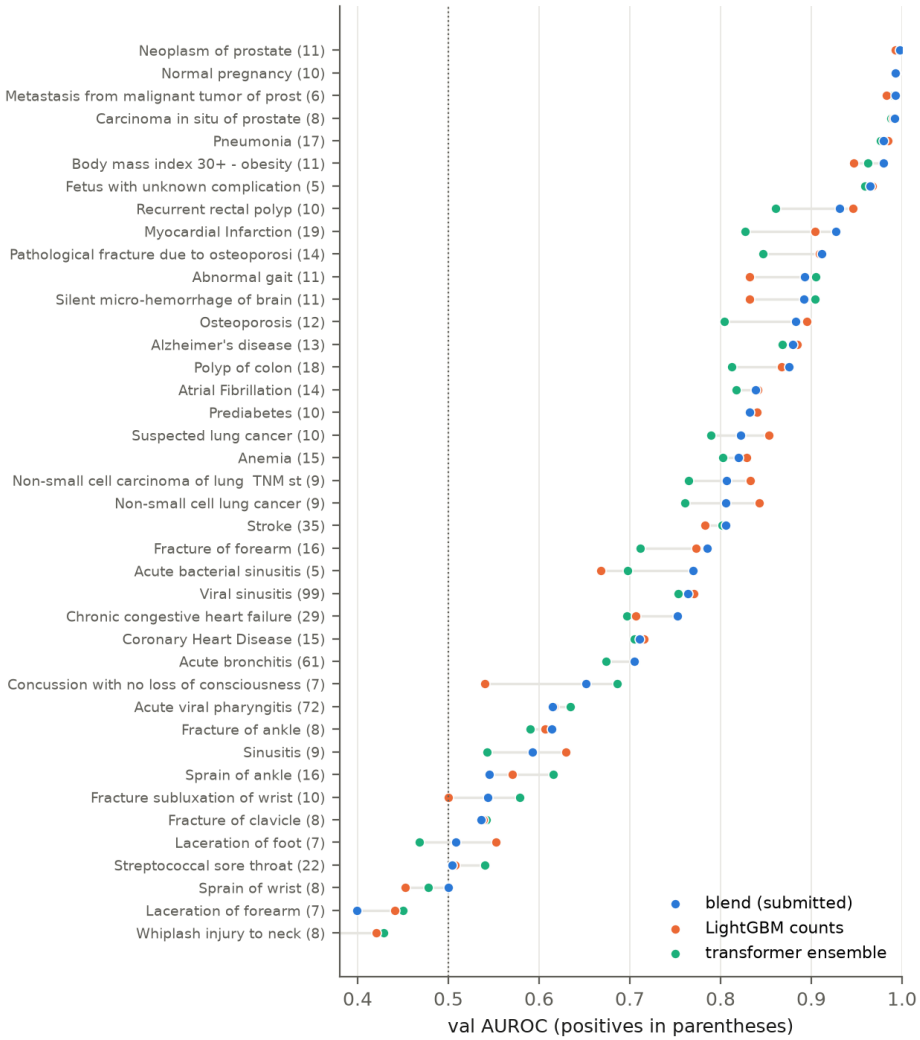


Figure 2: Official val AUROC per target for the blend, the transformer ensemble, and LightGBM on code counts, sorted by the blend. Numbers in parentheses are val positives; most targets have 5 to 20, so single-target differences are noisy.

of its range (Figure 3), and the Platt map corrects it. On val the blend's Brier score is 0.0365 against 0.0381 for the transformer alone.

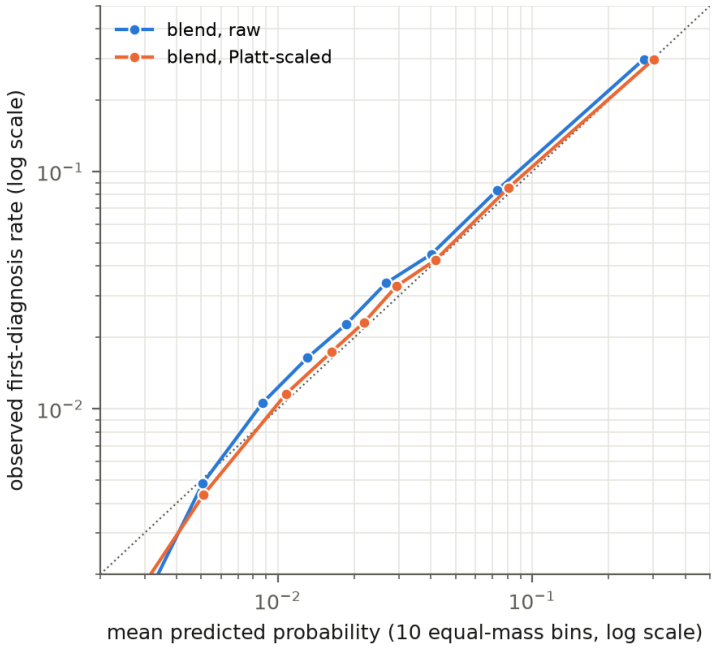


Figure 3: Reliability of the blend on training out-of-fold predictions, before and after Platt scaling, 10 equal-mass bins of all at-risk patient-target cells.

Limits. The data are synthetic and the anchor is retrospective, so nothing here is evidence about clinical use. Val is small: the macro AUROC interval is about 0.04 wide, and eleven targets have at most eight positives. Every CV number for the chosen configuration is the best of 28 transformer configurations (`models/transformer/`, counting seeds and pretraining parents) plus three stackers on the same five folds, so it is optimistic. The blend's mean-fold CV AUROC is 0.780 (pooled 0.774) against 0.767 on val, and choosing epochs on val instead of the training fold would have added 0.008 to the members (`reports/val/val_curves.csv`). The val scores are the first check and the hidden test the second. The LightGBM setting the blend uses sits on the edge of its search grid on all three axes (4 leaves, 100 trees, minimum child size 20), and I did not extend the grid. A simpler setting might score higher. I left it alone because any change would need a new budget, a refreeze, and a third val read. Two seeds of CV variation (about 0.002 AUROC in the mean, up to 0.014 per fold) are comparable to several of the differences in the tuning tables. A macro AUROC of 0.80 is out of reach on this data without the clock codes: the nine near-chance targets cap the average, and even an oracle choice between the transformer and LightGBM per target reaches 0.778 in CV.

5. Results summary and reproduction

Split	Model	Macro AUROC	mAP
train CV, pooled out of fold	blend (submitted), before Platt scaling	0.774	0.212
train CV, pooled out of fold	transformer ensemble	0.761	0.181

Split	Model	Macro AUROC	mAP
train CV, pooled out of fold	LightGBM counts	0.764	0.188
val (365), attempt 2	**blend (submitted)**	**0.767 [0.746, 0.788]**	**0.258 [0.234, 0.268]**
val, attempt 2	transformer ensemble	0.751	0.209
val, attempt 2	LightGBM counts	0.762 [0.740, 0.781]	0.239 [0.209, 0.249]
val, attempt 2	LightGBM counts + values	0.758 [0.740, 0.780]	0.257 [0.228, 0.267]
val, attempt 2	logistic regression, codes	0.764 [0.742, 0.782]	0.222 [0.195, 0.229]
val, attempt 2	age-sex incidence, at risk	0.727 [0.705, 0.751]	0.141 [0.125, 0.146]
val, attempt 1	transformer ensemble	0.751 [0.730, 0.773]	0.209 [0.192, 0.211]
test (358)	blend (submitted)	organisers' score	organisers' score

Paired on val, blend minus transformer ensemble: AUROC +0.016 [+0.002, +0.030], mAP +0.049 [+0.033, +0.072]. Blend minus LightGBM counts: AUROC +0.005 [-0.005, +0.016], mAP +0.019 [+0.002, +0.040]. The comparison models' own intervals were computed after the bundle from its saved prediction files (`reports/val/post_hoc_single_model_intervals.json`). The CV blend row is before Platt scaling; the per-fold maps change pooled scores (0.767 / 0.206) but no per-fold score. `predictions.csv` holds 358 rows by 40 code columns.

Reproduction: `make setup`, link the dataset to `data/raw`, then `make dataset`, `make modalities`, `make folds`, `make tokenize`, `make pretrain` and `make finetune` per fold and seed, `make lightgbm` and `make lightgbm-clean`, `make stacking`, `make calibration`, `make lightgbm-final`, `make freeze`, `make val-score`, `make predict`, `make submission`, `make package`. The Hugging Face package carries the 15 transformer checkpoints, 5 tokenizers, 80 LightGBM boosters with their feature specs, and the Platt weights; `python -m ehrforecast.hf_package verify` rebuilds `predictions.csv` from it on CPU and reproduced it exactly, in the development environment and in a fresh virtual environment built from `requirements-lock.txt`.